

May 24, 2026

textitI will introduce concepts like foas, hoas, phoas and verification condition generation. Then I will motivate using phoas for verification condition generation

1 Representing abstract syntax

When a program in a programming language is transforming the syntax of some other language, it often uses an abstract representation of the syntax in that programming language. Examples of this are compilers, interpreters, typecheckers and code generators. Since many languages have variables, the program transforming the syntax must have a way to deal with variable binding and scoping. Often this involves implementing substitution and finding ways to handle shadowing. This often can lead to bugs, for example in [?] a study was done on 9 DSL implementations and 8 of them were prone to variable capture. Some abstract representations make these variable bindings easier to deal with. I will discuss different representations of abstract syntax and discuss their advantages and disadvantages, focussing on different ways to handle variable bindings. In what follows I will call the language we're representing the syntax of the object language and the language we're representing the object language in the meta-language. As the examples, I will represent the object language of the untyped lambda calculus in the meta-language Haskell.

1.1 First Order Abstract Syntax (Foas)

In a first order abstract syntax, variables are represented with names.

1.1.1 Nominal names

The first first order approach is one that uses names nominally, I will discuss an approach that uses a **String** for these names.

The examples represent the identity function $(\lambda x.x)$ and function application $(\lambda f.\lambda x.fx)$

When a variable is introduced with a lambda, a name for it is given as a **String**. When a variable is used, this string is used to reference which introduction corresponds to which variable.

Advantages of this approach are that each variable has a unique and explicit identity. Additionally the name of a variable is the same everywhere it occurs.

```

| Lambda String Lam
| App Lam Lam

apply :: Lam
apply = Lambda "f" (Lambda "x" (App (Var "f") (Var "x")))

foasI :: Lam
foasI = Lambda "x" (Var "x")

```

Figure 1: Foas representation of the untyped lambda calculus

```

data DBTerm = DBVar Int
| DBLam DBTerm
| DBApp DBTerm DBTerm
deriving (Show,Eq)

deBruinI :: DBTerm
deBruinI = DBLam (DBVar 0)

apply :: DBTerm
apply = DBLam (DBLam (DBApp (DBVar 1) (DBVar 0)))

```

Figure 2: De Bruijn Index representation of the untyped lambda calculus

The syntax can easily be printed out and used for intentional analysis [?]. To represent syntax in this way, the meta-language does not require any advanced features such as higher order functions.

Some disadvantages of this approach are that it is possible to represent ill-formed syntax with this approach. For example, `Lamba "x" (Var "y")` can be respresented in this Foas approach, but the variable y was never introduced. Additionally there are more representations possible for alpha equivalent terms. For example, `Lamba "x" (Var "x")` and `Lamba "y" (Var "y")` represent the same function but have different representations.

1.1.2 De Bruijn Indices

The previous approach had the disadvantage that alpha-equivalent terms can have multiple representations. De Bruijn indecis try to solve this by giving no choice for what the name is when a variable is introduced. When a variable is used it is the natural number that measures the amount of lambda abstractions done between the introduction of the variable and the usage.

Advantages of this approach are that all alpha equivalent terms have a unique representation. Additionally each variable has a explicit identity. Just like the

```

| Lambda (Lam -> Lam)

hoasI :: Lam
hoasI = Lambda (\x->x)

apply :: Lam
apply = Lambda (\f -> Lambda (\x -> App f x))

```

Figure 3: Hoas representation of the untyped lambda calculus

nominal names foas approach, the syntax can easily be printed out, used for intentional analysis [?] and the meta-language does not need to support higher order functions.

A disadvantage of this approach is that you can the same variable might have different names throughout depending on where it occurs. For example the term $(\lambda x.x (\lambda f.x))$ would be represented as `DBLam (DBApp (Var 0) (DBLam (Var 1)))` where x is represented as both `Var 0` and `Var 1`. Another disadvantage would be that this representation can still create ill-formed terms. For example the term `DBLam (DBVar 7)` references a term that isn't introduced.

1.2 Higher order abstract syntax (HOAS)

Higher Order Abstract Syntax was introduced by [?] of representing abstract syntax in a meta-language supporting higher order functions. It uses the binding construct of the meta-language to represent the binding

1.3 Parametric Higher Order Abstract Syntax

(information from eg. Parametric Higher-Order Abstract Syntax for Mechanized Semantics)

1.4 Translations between representations

Describe translations $\text{phoas} \rightarrow \text{foas}$ and $\text{phoas} \rightarrow \text{hoas}$ such as described in Un-embedding Domain-Specific Languages.

2 monads?

(I'm not sure if this is necessary but it is something most other students would not have as background knowledge) Describe option, reader, state and continuation monads. yes fine, but be brief. Not 3 pages. Use it as a step up to explain specification monads

3 Verification condition generation

Describe what a verification condition is and how it can be computed by recursively calculating the weakest precondition.

3.1 Wstore and its combinators

Describe the Wstore monad and assert, demonic and angelic choice combinators.

3.2 generating verification conditions

Describe how to use these combinators to generate the weakest precondition.

(Verified Symbolic Execution with Kripke Specification Monads (and No Meta-programming))